

Viernes, 28 de Agosto del 2026



PROYECTO INTEGRADOR DE REDES NEURONALES

Mini-JARVIS: asistente de voz en español

sobre la arquitectura Transformer

Britany Tahis Macias Tapuy

Redes Neuronales

Índice

1. Antecedentes.....	3
1.1. Objetivos Principales.....	4
1.2. Herramientas Utilizadas	4
2. Proyecto Realizado (Mini-JARVIS: asistente de voz en español)	5
2.1. La arquitectura Transformer y el modulo de exploracion	5
2.2. Del Transformer al modelo de lenguaje	8
2.3. El recorrido de la voz: transcripcion, modelo y sintesis.....	9
2.4. La interfaz y los estados del asistente.....	10
2.5. Herramientas y funcionamiento sin internet.....	12
2.6. Decisiones de diseno	13
2.7. Limitaciones conocidas	14
2.8. Pruebas realizadas	15
3. Conclusiones y Recomendaciones	15
4. Referencias	17

1. Antecedentes

Este proyecto consiste en un asistente conversacional por voz en español, al que denomine Elena. El usuario mantiene pulsado un botón y habla; el sistema transcribe el audio, envía el texto a un modelo de lenguaje preentrenado junto con el contexto de la conversación, recibe una respuesta escrita, la convierte en audio y la reproduce. Todo ocurre dentro de una ventana que indica en cada momento la etapa en la que se encuentra el sistema.

El trabajo articula dos planos. Por un lado la teoría de la arquitectura Transformer y de los modelos de lenguaje, estudiada en la asignatura. Por otro la integración de un modelo real dentro de una aplicación estable, que funcione de extremo a extremo y resuelva los fallos previsibles sin interrumpirse. No entrene ningún modelo, porque el enunciado no lo requiere y no resulta viable en el tiempo de un curso. En cambio sí inspeccione uno internamente, con el fin de poder explicar el tratamiento que recibe una frase desde que ingresa hasta que se produce la respuesta.

El código fuente reside en un repositorio público de GitHub con 77 commits distribuidos por tareas, de modo que el historial refleja el avance y no un único envío final. De forma

complementaria mantengo una carpeta de control interno donde quedan registrados el plan de trabajo, las pruebas y las decisiones adoptadas, incluidas las alternativas descartadas.

1.1. Objetivos Principales

- Explicar y evidenciar, a partir de la salida real del propio programa, los conceptos de tokenización, embeddings, positional encoding y self-attention.
- Integrar un modelo de lenguaje preentrenado en una aplicación funcional de extremo a extremo, tanto mediante API en la nube como en ejecución local.
- Implementar el pipeline de voz completo: reconocimiento de voz, procesamiento con el modelo y síntesis de voz.
- Diseñar una interfaz que comunique de forma inequívoca el estado del asistente, y aplicar buenas prácticas de ingeniería de software: manejo de errores, gestión segura de credenciales, control de versiones y documentación técnica.
- Analizar las limitaciones, los sesgos y las implicaciones éticas del uso de un modelo de lenguaje en una aplicación destinada a terceros.

1.2. Herramientas Utilizadas

El proyecto está desarrollado en Python 3.14 sobre un entorno virtual, con todas las dependencias fijadas por versión para garantizar que una instalación en otra máquina reproduzca el mismo entorno.

El reconocimiento de voz emplea Whisper large-v3 (Radford et al., 2023), servido por Together AI. El procesamiento del lenguaje recae en dos modelos intercambiables desde la propia interfaz: Qwen3.8-2.4T-A95B como modelo principal y Llama-3.3-70B-Instruct-Turbo como

alternativo. La síntesis de voz utiliza edge-tts, cliente del servicio de síntesis de Microsoft, con la voz es-AR-ElenaNeural.

Para el funcionamiento sin conexión, esas tres piezas disponen de equivalentes que se ejecutan en la máquina local: faster-whisper para la transcripción, un modelo Qwen2.5 de 0.5B para la generación de respuestas y la voz del sistema operativo para la síntesis.

El módulo de exploración utiliza BETO (Canete et al., 2020), un modelo de aproximadamente 110 millones de parámetros entrenado únicamente en español, cargado mediante Hugging Face Transformers (s.f.) y PyTorch, con matplotlib para la representación gráfica. La interfaz está construida con CustomTkinter (Schimansky, s.f.). Las herramientas del asistente se apoyan en psutil para consultar el estado del equipo, ddgs para las búsquedas web y httpx para la consulta meteorológica. Las pruebas automáticas se ejecutan con pytest.

Conviene justificar una elección que puede resultar llamativa: la comunicación con Together AI se realiza mediante la librería de OpenAI. Together implementa una API compatible, de manera que basta con modificar la dirección del servidor. Desarrollar un cliente propio habría supuesto un esfuerzo adicional sin ninguna ventaja técnica.

2. Proyecto Realizado (Mini-JARVIS: asistente de voz en español)

2.1. La arquitectura Transformer y el módulo de exploración

Un modelo de lenguaje no procesa letras. La primera operación que se aplica a una frase es la tokenización: el texto se divide en unidades discretas denominadas tokens, que pueden corresponder a una palabra completa, a un fragmento de palabra o a un signo de puntuación, y cada unidad se sustituye por un identificador numérico. Las palabras ausentes del vocabulario del

modelo se fragmentan en varias piezas. En el modulo de exploracion, la palabra Wikipedia se descompone en seis tokens.

Cada token se transforma a continuacion en un vector de alta dimensionalidad, denominado embedding, que constituye la representacion del significado con la que opera el modelo. Ese vector, sin embargo, no contiene informacion sobre la posicion que la palabra ocupaba en la frase, y el mecanismo de atencion tampoco la deduce, puesto que no procesa la secuencia en orden. Esa carencia la resuelve el positional encoding, una senal dependiente de la posicion que se suma a cada token.

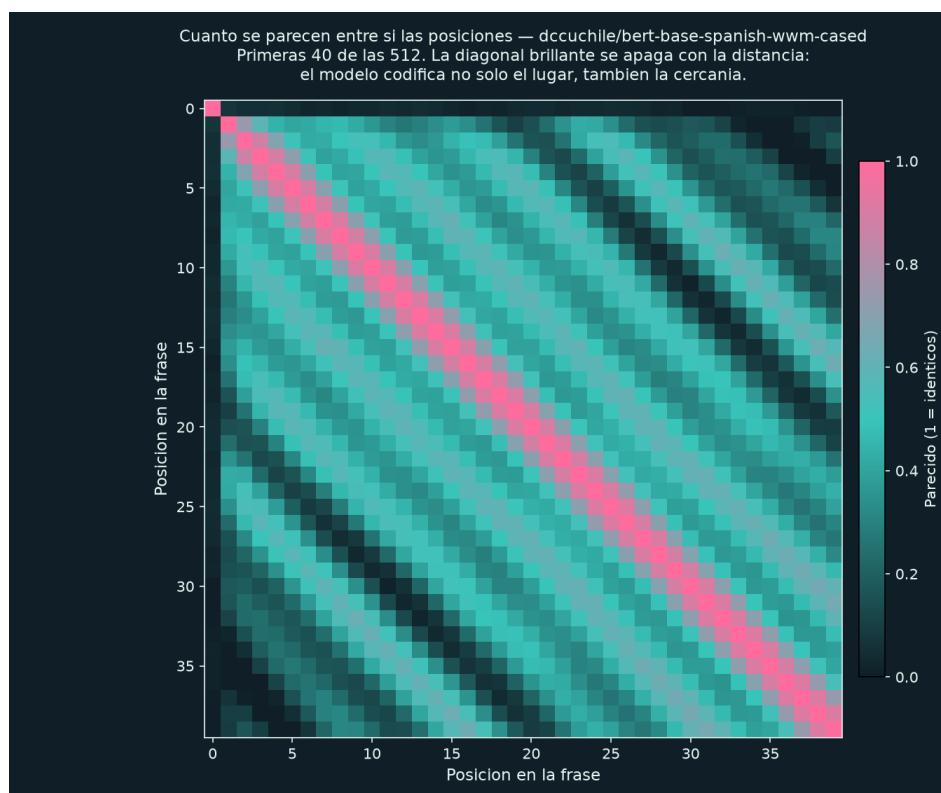


imagen 1 Similitud entre posiciones. La diagonal se atenúa con la distancia, lo que indica que el modelo codifica tanto el lugar como la proximidad.

Sobre esa representacion opera el self-attention, mecanismo central de la arquitectura Transformer (Vaswani et al., 2017). Cada token pondera su relacion con todos los demas de la

secuencia y construye así una representación contextual. De ello se deriva que una misma palabra adquiera significados distintos según el contexto: banco de río y banco de la calle comparten grafía, y lo que las diferencia es el conjunto de palabras al que cada una atiende. En la representación gráfica, cada fila corresponde a un token y la intensidad indica el peso asignado a los demás. Cada fila suma exactamente uno, ya que constituye una distribución de probabilidad.

Esta operación no se ejecuta una sola vez. El mecanismo se denomina multi-head attention porque el modelo realiza varias lecturas paralelas de la misma secuencia, y cada cabeza aprende a atender a un tipo distinto de relación lingüística: la concordancia entre sujeto y verbo, la referencia de un adjetivo a su sustantivo, entre otras. El modelo empleado en el módulo de exploración cuenta con doce capas de doce cabezas cada una, lo que produce ciento cuarenta y cuatro mapas de atención distintos para una misma frase, seleccionables desde la interfaz.

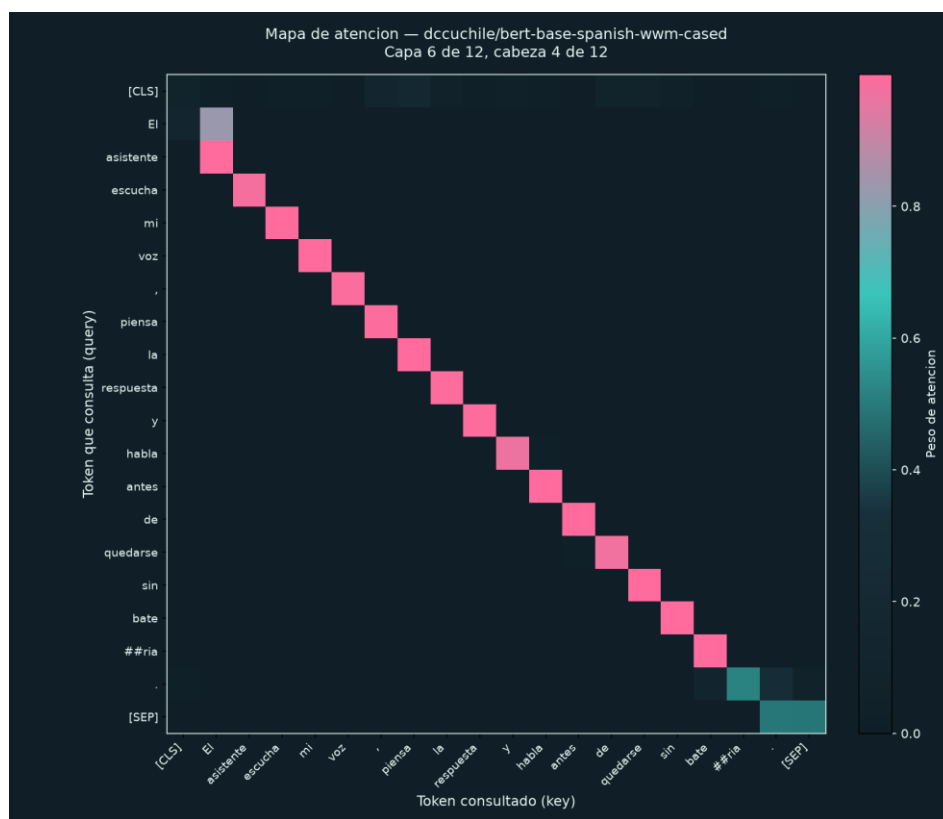


imagen 2 Mapa de self-attention correspondiente a una frase de ejemplo.

El modulo de exploracion no emplea el mismo modelo que la conversacion, y la razon es tecnica. El modelo con el que dialoga Elena pertenece a un proveedor externo y su acceso se limita a una API que devuelve texto, por lo que no existe forma de obtener sus pesos de atencion. BETO, en cambio, se descarga y puede inspeccionarse.

Durante el desarrollo se presento un problema que estuvo cerca de invalidar el modulo completo. Al solicitar los pesos de atencion, el resultado devuelto era vacio y no se generaba ninguna excepcion. La causa reside en que la implementacion de atencion que el modelo carga por defecto no materializa esos pesos, y es necesario solicitar explicitamente la implementacion clasica. Dado que el fallo era silencioso, el codigo incorpora ahora una comprobacion que verifica la existencia de los pesos y notifica el error si vuelven a llegar vacios.

2.2. Del Transformer al modelo de lenguaje

La arquitectura Transformer original constaba de dos componentes. Un encoder, que procesa la secuencia completa y construye su representacion atendiendo simultaneamente a ambos lados de cada token, y un decoder, que genera texto de forma secuencial y solo puede atender a lo ya producido. Este proyecto emplea ambos. El modulo de exploracion utiliza BETO, de tipo encoder-only, orientado a la comprension, motivo por el cual sus mapas de atencion resultan interpretables en ambas direcciones. La conversacion, en cambio, recae en un modelo decoder-only autoregresivo, arquitectura predominante en los asistentes conversacionales actuales, dado que responder exige generar texto y no unicamente interpretarlo.

La generacion autoregresiva consiste en producir un token, incorporarlo a la propia entrada y generar el siguiente a partir del conjunto resultante. Tres parametros gobiernan esa seleccion. La

temperatura regula el grado de aleatoriedad: valores proximos a cero conducen a elegir siempre el token mas probable, con respuestas predecibles, mientras que valores altos amplian el espacio de eleccion y aumentan tanto la originalidad como la probabilidad de error. El top_p opera de forma complementaria, restringiendo la seleccion a los candidatos mas probables hasta acumular un porcentaje determinado. El maximo de tokens limita la extension de la respuesta, parametro relevante en este proyecto porque toda respuesta se reproduce en voz alta.

Un modelo de lenguaje atraviesa varias etapas antes de poder conversar. El preentrenamiento lo expone a grandes volumenes de texto y le ensena unicamente a continuar una secuencia. Un modelo que se detiene en esa fase se denomina modelo base y, ante una pregunta, tiende a prolongar el texto en lugar de responderlo. El fine-tuning consiste en continuar el entrenamiento con datos de un dominio concreto para especializarlo. El instruction-tuning es una variante de esa etapa que resulta determinante en este caso: ensena al modelo que los enunciados con forma de pregunta o de instruccion deben responderse. Los modelos empleados incorporan el termino Instruct en su identificador precisamente por haber atravesado esa fase. Sin ella el system prompt careceria de efecto, ya que un modelo base no obedece instrucciones sino que las continua.

El system prompt es el unico elemento que define la personalidad del asistente. Establece su nombre, su condicion de sistema automatico y el estilo de redaccion exigido: frases breves, sin listas ni simbolos de formato, dado que el texto se convierte en audio.

2.3. El recorrido de la voz: transcripcion, modelo y sintesis

El codigo esta organizado por responsabilidades. Un paquete para el nucleo, que agrupa la captura de audio, la transcripcion, el motor de lenguaje, la memoria conversacional y el orquestador encargado de coordinar cada turno. Otro para la interfaz grafica, otro para las herramientas y otro

para el modulo de exploracion. Un unico archivo de configuracion centraliza todos los parametros ajustables.

La grabacion se produce mientras el boton permanece pulsado y, al liberarlo, el audio se envia a transcribir. La grabacion no se escribe en disco en ningun momento: permanece en memoria, se transcribe y se descarta.

El texto transcrito se envia al modelo acompañado del system prompt y de los ultimos turnos de la conversacion. En este punto interviene la ventana de contexto, que delimita la cantidad maxima de texto que el modelo puede procesar en una sola peticion. El modelo no conserva informacion entre llamadas sucesivas; la continuidad aparente de la conversacion se obtiene reenviando el historial completo en cada turno. Puesto que ese historial no puede crecer indefinidamente sin exceder la ventana ni incrementar el coste, el sistema conserva los diez ultimos turnos y descarta el mas antiguo. Si la conversacion se prolonga, el asistente no falla: unicamente pierde la informacion inicial, y un indicador en la interfaz advierte de ello antes de que ocurra.

La respuesta generada se convierte en audio y se reproduce. El estilo breve impuesto por el system prompt responde a esta ultima etapa, ya que los elementos de formato carecen de equivalente hablado.

2.4. La interfaz y los estados del asistente

La ventana se organiza en dos columnas. La izquierda contiene un indicador circular que varia segun el estado del sistema; la derecha, la transcripcion de la conversacion y los controles. Los estados son cuatro, mas el de error, y cada uno modifica el color y el movimiento del indicador, de manera que la etapa en curso resulta identificable sin necesidad de leer.

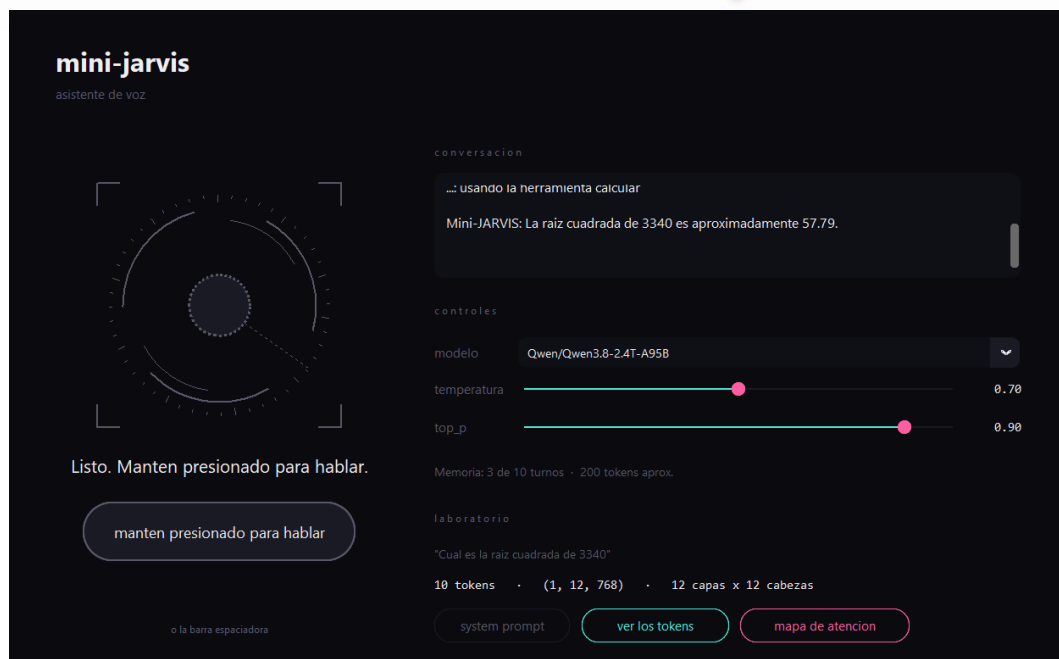


imagen 3 Interfaz en estado de reposo, con los controles y el modulo de exploracion en la columna derecha.

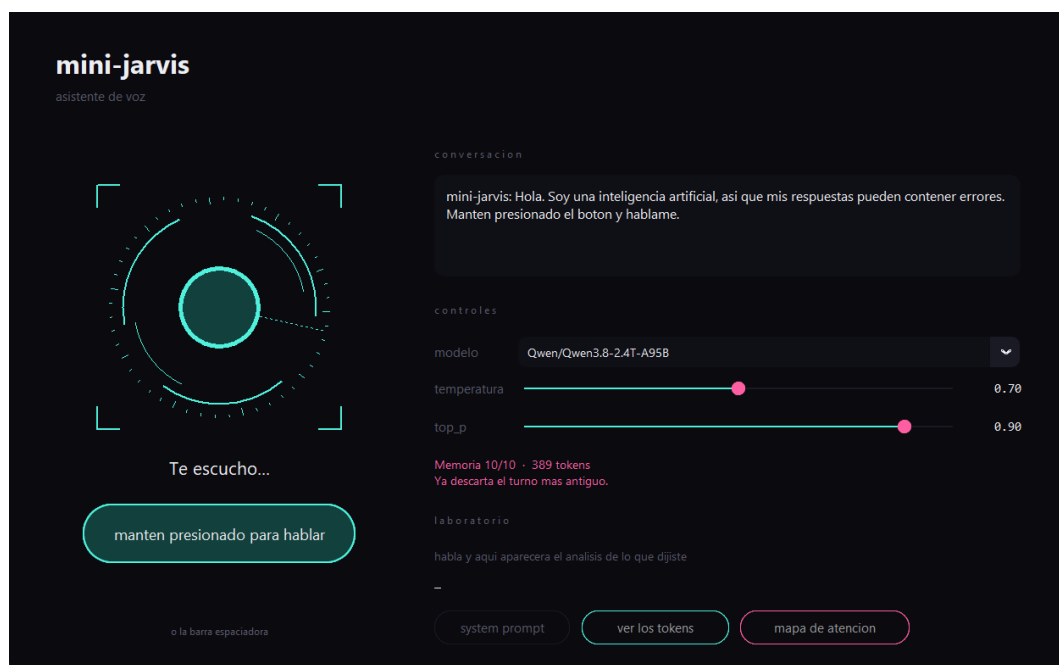


imagen 4 Estado de escucha: el indicador responde durante la grabación.

Bajo la conversacion se disponen controles orientados a la demostracion de los conceptos teoricos: un selector de modelo, deslizadores de temperatura y top_p, y un contador de memoria que advierte cuando el turno mas antiguo va a descartarse. El deslizador de temperatura alcanza un

maximo de 1.4 y no de 2 por una razon medida: en las pruebas contra la API, a partir de 1.5 el modelo de razonamiento se bloqueaba durante aproximadamente cien segundos en dos de cada tres intentos.

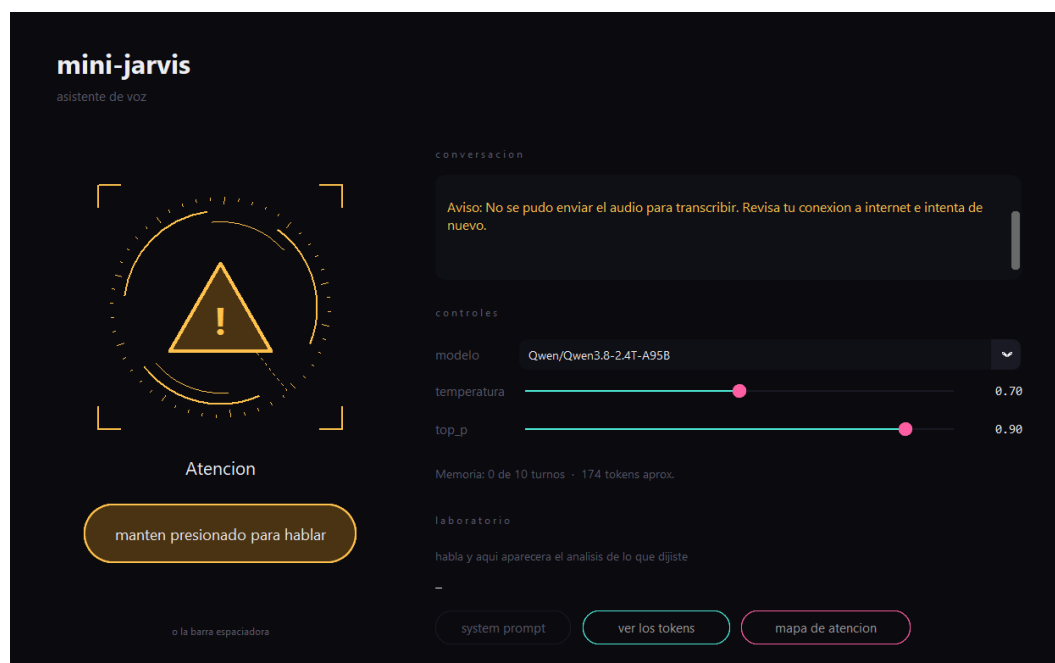


imagen 5 Mapa de atencion generado desde la propia aplicacion.

Los fallos previsibles estan contemplados y ninguno interrumpe la ejecucion: ausencia de conexion, audio no reconocido, respuesta vacia, credencial incorrecta y tiempo de respuesta excesivo. Cada situacion dispone de un mensaje especifico, dado que un mensaje que atribuye el fallo a una causa equivocada resulta mas perjudicial que la ausencia de mensaje. En cuanto a accesibilidad, la interaccion admite tambien la barra espaciadora ademas del boton, y el estado nunca se comunica unicamente mediante color: siempre lo acompaña una indicacion textual.

2.5. Herramientas y funcionamiento sin internet

El asistente no se limita a conversar, sino que ejecuta acciones sobre el sistema mediante tool calling. El modelo recibe la relacion de operaciones disponibles y, cuando la peticion lo

requiere, responde solicitando una de ellas en lugar de generar texto. El sistema dispone de diez: calculo aritmetico, consulta de hora, consulta meteorologica, estado del equipo, busqueda web, apertura de paginas, control de volumen, control de brillo, apertura de carpetas y reproduccion de video.

Toda operacion que actua sobre el sistema atraviesa previamente una lista blanca. La calculadora, por ejemplo, no recurre a la funcion de evaluacion de Python, que ejecutaria cualquier expresion generada por el modelo; en su lugar analiza sintacticamente la expresion y admite exclusivamente operaciones aritmeticas. Las paginas y las carpetas accesibles proceden igualmente de una relacion cerrada. El modelo propone la accion, pero la decision sobre su ejecucion corresponde al programa.

Aunque el enunciado no lo exige, el asistente conserva su funcionalidad sin conexion. Ante una caida de red, las tres piezas del pipeline conmutan automaticamente a sus equivalentes locales y la interfaz senala el cambio de modo. La calidad de las respuestas disminuye de forma apreciable, dado que el modelo local es aproximadamente mil veces menor que el remoto. A cambio, el sistema sigue operativo, mientras que sin este mecanismo una perdida de conexion lo inutilizaba por completo.

La conmutacion distingue entre una caida de red y una credencial mal configurada. En el segundo caso el sistema no cambia a modo local, puesto que hacerlo ocultaria un error de configuracion en lugar de exponerlo.

2.6. Decisiones de disenio

Cada turno de conversacion se ejecuta en un hilo independiente que se crea y finaliza con el propio turno. La alternativa consistia en construir el sistema sobre asyncio, opcion tecnicamente

mas elegante, pero la biblioteca de interfaz grafica no es asincrona y su combinacion exige un puente que introduce fallos dificiles de reproducir. Un hilo por turno resulta mas simple de comprender y de justificar.

La eleccion de dos modelos de familias distintas, en lugar de dos tamanos de la misma, responde a una incidencia real. A tres dias de la entrega, el proveedor retiro el modelo alternativo configurado. La verificacion de los 169 modelos del catalogo revelo que unicamente veinte respondian. Si ambos modelos pertenecen a la misma familia, una incidencia del proveedor puede afectarlos simultaneamente.

La memoria conversacional se trunca por turnos y no por tokens. El recuento exacto de tokens requiere el tokenizador del proveedor y depende de la conexion, mientras que el recuento por turnos puede probarse sin red y visualizarse en la interfaz. La decision sacrifica precision a cambio de un comportamiento verificable y demostrable.

2.7. Limitaciones conocidas

La limitacion mas relevante es la generacion de contenido incorrecto. El modelo puede producir un dato falso y expresarlo con el mismo grado de seguridad que uno verificado. No se trata de un defecto del programa sino de una caracteristica inherente al funcionamiento de los modelos de lenguaje, por lo que no admite correccion mediante codigo. La medida adoptada es informativa: el asistente advierte al iniciarse de su naturaleza automatica y de la posibilidad de error, y la misma advertencia figura en su system prompt.

El rendimiento sin conexion es sensiblemente inferior. El modo con conexion depende de la disponibilidad de saldo en la API y de la latencia de red, de modo que una respuesta puede demorarse varios segundos. El modelo principal es cerrado: no es posible examinar sus pesos ni

conocer los datos con los que fue entrenado, lo que impide auditar sus sesgos. La transcripción pierde fiabilidad en presencia de ruido ambiental o cuando la distancia al microfono es excesiva, y en tales casos el modelo responde a un enunciado que el usuario no formulo. Por ultimo, el sistema no identifica a su interlocutor, por lo que no resulta apropiado para aplicaciones que requieran autenticacion de personas.

2.8. Pruebas realizadas

El proyecto cuenta con 189 pruebas automaticas que se ejecutan en aproximadamente doce segundos sin requerir conexion, microfono ni consumo de la API. Cubren el analisis de las respuestas del modelo, la memoria conversacional y su truncado, las diez herramientas, el tratamiento de cada tipo de error y la conmutacion a modo local.

Existe ademas un conjunto de verificaciones que no admiten automatizacion, por tratarse de comprobaciones perceptivas: la naturalidad de la voz sintetizada, la distinguibilidad de los estados en la interfaz y la aparicion del aviso de modo local al desactivar la conexion. Estas se realizaron manualmente y quedan registradas. Se verificaron asimismo los tres escenarios principales: una conversacion de varios turnos consecutivos, una peticion que obliga al uso de una herramienta y una interrupcion de red durante el uso.

3. Conclusiones y Recomendaciones

El desarrollo confirma que es posible construir un asistente de voz completo sobre un modelo preentrenado sin necesidad de entrenar ninguno. La integracion de los componentes resulto ser la parte menos exigente del trabajo; la comprension del funcionamiento interno de cada uno requirio un esfuerzo considerablemente mayor, y es en ese punto donde el modulo de exploracion

aporto el mayor valor formativo, al obligar a examinar el modelo en lugar de tratarlo como una caja opaca.

La decision de mayor alcance fue mantener el orquestador desacoplado del resto de componentes. Esa separacion, adoptada inicialmente por claridad, permitio incorporar posteriormente el modo sin conexion sin modificar ninguna linea de la logica conversacional: fue suficiente con envolver las tres piezas que se comunican con el servicio remoto.

En el plano etico, la conclusion es que no toda limitacion de un sistema de este tipo admite solucion tecnica. Cuando el problema reside en el comportamiento del modelo y no en el codigo que lo rodea, la unica respuesta responsable consiste en informar al usuario y documentarlo, en lugar de presentar el sistema como mas fiable de lo que es.

A partir de la experiencia obtenida se formulan las siguientes recomendaciones.

- Incorporar una palabra de activacion que evite la dependencia del boton. Es el unico requisito opcional del enunciado que no llego a cubrirse.
- Sustituir el modelo local por uno de mayor tamano cuando las condiciones del equipo lo permitan, con el fin de reducir la diferencia de calidad entre ambos modos de funcionamiento.
- Verificar la disponibilidad de saldo en la API antes de cualquier demostracion, dado que el modo con conexion depende de ella.
- Revisar periodicamente la vigencia de los modelos seleccionados, puesto que durante el desarrollo un proveedor retiro uno de ellos con tres dias de antelacion.

4. Referencias

Canete, J., Chaperon, G., Fuentes, R., Ho, J.-H., Kang, H., y Perez, J. (2020). Spanish pre-trained BERT model and evaluation data. PML4DC at ICLR 2020.

<https://github.com/dccuchile/beto>

Hugging Face. (s.f.). Transformers documentation. Recuperado el 28 de agosto de 2026, de

<https://huggingface.co/docs/transformers>

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., y Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. Proceedings of the 40th International Conference on Machine Learning. <https://arxiv.org/abs/2212.04356>

rany2. (s.f.). edge-tts [Software]. GitHub. Recuperado el 28 de agosto de 2026, de

<https://github.com/rany2/edge-tts>

Schimansky, T. (s.f.). CustomTkinter documentation. Recuperado el 28 de agosto de 2026, de

<https://customtkinter.tomschimansky.com/>

SYSTRAN. (s.f.). faster-whisper [Software]. GitHub. Recuperado el 28 de agosto de 2026, de

<https://github.com/SYSTRAN/faster-whisper>

Together AI. (s.f.). Together AI documentation. Recuperado el 28 de agosto de 2026, de

<https://docs.together.ai/>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., y

Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information

Processing Systems, 30. <https://arxiv.org/abs/1706.03762>